



Carátula proyecto FINAL

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martinez

Asignatura: Estructura de Datos y Algoritmos I

Grupo: 5

No de Práctica(s): SIMULADOR DE SISTEMA OPERATIVO

Integrante(s):

Roque Robledo Fernando Ignacio

Jasso Olivares Germán

Hernandez Piedra José Enrique

Hernández Beltran Ena Elianeth

EQUIPO TRALARELITOS

Semestre: 2do

Fecha de entrega: 11/06/2026

OBJETIVO

Crear un simulador en lenguaje C de un sistema operativo, encargándose de acomodar procesos en la memoria y decidir en qué orden se van a ejecutar. Además, implementar el uso de Python para generar datos de prueba automáticamente y hacer gráficas que nos dejen ver cuál combinación de algoritmos es la más rápida y eficiente.

INTRODUCCIÓN

Los sistemas operativos como Windows, macOS o Android tienen que trabajar con algoritmos todo el tiempo: coordinar qué programas se ejecutan primero en el procesador y asegurarse de que cada uno tenga su propio espacio en la memoria RAM para trabajar sin chocar con los demás.

Para entender cómo funcionan estos dos procesos tan importantes, en este proyecto creamos un Simulador de Sistema Operativo. El núcleo del simulador está programado en lenguaje C, que es el esencial en cualquier programa como este para controlar la memoria y los procesos de forma muy directa y rápida.

Por otro lado si se llegara a probar el simulador metiendo datos a mano uno por uno sería muy tardado y por ese motivo, usamos el lenguaje de programación en Python. Usamos este lenguaje para crear "scripts" (programas automáticos) que inventan cientos de procesos de prueba con diferentes tamaños y tiempos de ejecución. Además, Python se encarga de guardar los resultados en tablas organizadas y crear gráficas para que podamos comparar visualmente qué métodos funcionan mejor y cuáles vuelven más lento al sistema. Con esto, logramos ver en la práctica toda la teoría que aprendimos en la materia de Estructura de Datos y Algoritmos.

MARCO TEÓRICO

Para entender cómo funciona nuestro simulador, primero debemos conocer los conceptos básicos de cómo un sistema operativo real organiza el trabajo y la memoria de la computadora.

Conceptos del Sistema Operativo

Proceso

En este proyecto, un proceso es una tarea simulada que necesita tiempo de procesador y un espacio en la memoria RAM para poder ejecutarse. Cada proceso tiene una tarjeta de identificación con los siguientes datos:

PID: El número de identificación único del proceso.

Burst Time: El tiempo total que el proceso necesita usar el CPU para terminar su trabajo.

Remaining Time: El tiempo que le falta por ejecutarse (se va reduciendo conforme trabaja).

Priority: Qué tan importante es el proceso.

Memory Required: El espacio de memoria RAM que pide para poder funcionar.

Estados de los Procesos

Cuando los procesos van cambiando de estado según lo que están haciendo:

READY: Está formado en la cola, esperando su turno para usar el CPU.

RUNNING (En ejecución): Está usando el CPU en ese preciso instante.

BLOCKED (Bloqueado): Está pausado o esperando a que pase algo para poder continuar.

FINISHED (Terminado): El proceso ya completó todo su tiempo de trabajo y liberó sus recursos.

Quantum

Es el tiempo máximo continuo que el planificador (Scheduler) le presta el CPU a un proceso. Por ejemplo, si el Quantum es de 2 segundos y el proceso necesita 5, el CPU lo interrumpe a los 2 segundos, lo manda al final de la fila y pasa al siguiente proceso para que todos avancen de forma justa.

Gestión y Administración de Memoria

El simulador tiene un Administrador de Memoria que empieza con un único bloque libre gigante (toda la memoria disponible). Conforme llegan los procesos, este bloque se va partiendo.

Fragmentación de Memoria

Ocurre cuando la memoria se llena y se vacía en diferentes pedazos. Al final, te quedan muchos "huecos" libres pero están separados por procesos en medio. Si llega un proceso nuevo grande, no cabe porque no hay un solo bloque continuo que tenga el tamaño que pide, aunque la suma de todos los huequitos sea suficiente.

Coalescencia y Compactación

Para solucionar la fragmentación, usamos dos técnicas:

Coalescencia: El programa revisa la memoria y, si encuentra dos o más bloques vacíos que están juntos (vecinos), los junta para hacer un solo bloque más grande.

Compactación: Es como acomodar los bloques del Tetris; mueve todos los procesos que están activos hacia un lado de la memoria para dejar todo el espacio libre unificado del otro lado.

ESTRUCTURAS DE DATOS UTILIZADAS

Para que el simulador funcione ordenadamente, usamos diferentes estructuras de datos lineales en lenguaje C:

Pila (Stack)

Para qué se usó: Se encarga de guardar el historial de los estados de ejecución de los procesos y nos ayuda en el algoritmo de *Backtracking* para regresar a un punto anterior si ocurre algún problema.

Cola Simple (Queue)

Para qué se usó: Es una fila normal (el primero que llega es el primero que se atiende). La usamos obligatoriamente para controlar el planificador de procesos FIFO.

Cola Circular (Circular Queue)

Para qué se usó: Es una fila donde el último elemento se conecta otra vez con el primero. Es la estructura obligatoria para el planificador Round Robin, ya que permite que los procesos que no han terminado vuelvan al inicio de la fila de forma cíclica.

Lista Ligada Simple (Linked List)

Para qué se usó: Es una cadena de nodos conectados en una sola dirección. La usamos específicamente para guardar y registrar de forma ordenada, según su PID, todos los procesos que ya están FINISHED.

Lista Doblemente Ligada (Doubly Linked List)

Para qué se usó: Es una lista donde cada nodo conoce a su vecino de adelante y al de atrás. Es perfecta para el mapa de memoria dinámica, ya que nos permite caminar en ambas direcciones para detectar bloques vacíos contiguos y hacer la coalescencia de manera eficiente.

ESTRATEGIAS DE ALGORITMOS Y SU IMPLEMENTACIÓN

La primera área es la asignación de memoria, donde el sistema analiza un mapa de bloques mediante una lista doblemente ligada utilizando tres estrategias diferentes cuando un proceso nuevo solicita espacio disponible.

El método First Fit, o primer ajuste, asigna el proceso al primer espacio vacío que encuentra en el mapa. Destaca por ser la alternativa más rápida al evitar recorridos extensos, aunque suele dejar pequeños espacios libres desaprovechados al inicio de la estructura.

El método Best Fit, o mejor ajuste, escanea mediante fuerza bruta todos los bloques vacíos para seleccionar el que se adapte de forma más exacta al tamaño solicitado. Su ventaja es que minimiza el desperdicio de memoria, pero vuelve el sistema más lento debido al análisis exhaustivo.

El método Worst Fit, o peor ajuste, realiza la operación inversa al ubicar el proceso en el bloque vacío de mayor tamaño disponible. El objetivo de esta estrategia es garantizar que el espacio sobrante sea lo bastante grande para alojar futuras tareas sin fragmentar el entorno de inmediato.

La segunda área es la planificación de procesos, la cual se encarga de coordinar qué entidad tiene derecho a utilizar el procesador en cada instante para avanzar con su ejecución.

El primer planificador es FIFO, que funciona como una fila convencional por orden de llegada. El proceso inicial ocupa el procesador hasta terminar todo su trabajo, lo que simplifica su programación pero genera el efecto convoy, retrasando drásticamente a las tareas cortas que esperan detrás.

El segundo planificador es SJF, una estrategia ávida que prioriza y ejecuta primero al proceso con el menor tiempo restante de trabajo. Este modelo es el óptimo para reducir el tiempo de espera general, pero puede provocar inanición si el flujo constante de tareas cortas impide que las grandes se ejecuten.

El tercer planificador es Round Robin, el cual distribuye el uso del procesador de forma equitativa otorgando a cada tarea un tiempo límite llamado quantum. Si el tiempo expira y el trabajo no ha concluido, el proceso regresa a estado listo y se mueve al final de una cola circular de manera cíclica.

Como complemento del núcleo, se incluyeron enfoques avanzados dentro del directorio de algoritmos para resolver problemáticas complejas de optimización.

El archivo de backtracking busca combinaciones de alojamiento de forma recursiva, teniendo la capacidad de regresar un paso atrás en la estructura de la pila si detecta que la ruta elegida satura el sistema de manera ineficiente.

El módulo de divide y vencerás se utiliza para acelerar el ordenamiento de la cola de procesos mediante la técnica de Merge Sort basándose en prioridades antes de transferir los datos al planificador principal.

Finalmente, las opciones de programación dinámica resuelven matemáticamente la distribución de espacio modelando el entorno como el problema de la mochila, almacenando las soluciones en una matriz para evitar la repetición de cálculos.

COMPLEJIDAD COMPUTACIONAL Y RECURRENCIAS

El uso de la notación Big O permite evaluar el rendimiento en tiempo y el uso de memoria RAM de cada módulo de nuestro simulador.

El archivo de divide y vencerás para ordenar procesos se resuelve mediante el segundo caso del Teorema Maestro, dando una complejidad temporal de $N \log N$ y una espacial lineal de N .

Los archivos de programación dinámica para optimizar el espacio aplican el modelo de la mochila con una matriz de estados, determinando una complejidad temporal de N por C y una espacial de N por C .

El módulo de backtracking evalúa recursivamente las combinaciones de memoria, generando un árbol de decisiones cuya solución por sustitución revela una complejidad temporal exponencial de B a la N y una espacial lineal de N .

Las estructuras base como pilas, colas simples y colas circulares operan bajo un tiempo constante de tipo 1 para insertar y extraer procesos, manteniendo un consumo de memoria lineal equivalente a N .

La administración de memoria sobre la lista doubly linked requiere un tiempo lineal de tipo B , ya que la búsqueda de espacios libres y la coalescencia exigen recorrer secuencialmente los bloques existentes.

Los planificadores presentan un tiempo lineal de tipo N en FIFO por su despacho secuencial, una complejidad cuadrática de N al cuadrado en SJF por sus búsquedas continuas, y un costo de Q por N en Round Robin según el quantum.

CONCLUSIÓN

El desarrollo de este simulador permitió constatar de forma práctica cómo la correcta selección de estructuras de datos lineales y algoritmos de asignación impacta directamente en el rendimiento de un sistema operativo. La experimentación demostró que mientras los enfoques ávidos como *First Fit* optimizan los tiempos de respuesta iniciales, requieren de mecanismos complementarios como la coalescencia para mitigar la fragmentación degradation.

Asimismo, el planificador *Round Robin* demostró ser el modelo más equitativo para entornos de alta concurrencia al garantizar un reparto justo del procesador mediante el control estricto del *quantum*.

Por otra parte, la metodología de integración de lenguajes evidenció la potencia de combinar la velocidad de ejecución de bajo nivel en C con la versatilidad de Python para el procesamiento automatizado de datos y generación de gráficos. Finalmente, el uso guiado y crítico de la inteligencia artificial generativa sirvió como un catalizador para resolver incompatibilidades de compilación y optimizar la arquitectura modular del código, consolidando los aprendizajes analíticos de complejidad computacional y sentando las bases esenciales del diseño de software en ingeniería.